Frontiers: A Standardized Protocol for Text-Based Construct Classification Using Large Language Models

Online Appendix: Supplemental Methodological Details and Results

Table of Contents

Online Appendix A: Web Application Implementation

We outline the key steps a user needs to follow in the web application. We also provided a 5 minutes video tutorial (https://youtu.be/HVcKLRiilmQ?si=GCXIK8Cl8QwkTwpq). Setup: The user first obtains API access to LLMs via a platform like OpenRouter (openrouter.ai). After securing the necessary API key (for example, an OpenAI GPT-4.1 key and access to another model), the user launches the web app in a browser (no local installation required). The user enters their API credentials into the interface to enable the LLMs.

1. Data Upload: The user prepares a CSV file containing the text data to be analyzed, where each text instance (e.g., a sentence, review, or social media post) is in a separate row (with any identifier or metadata fields as needed). The app provides prompts to ensure the data is in the correct format, and the user uploads the file via the interface.

2. Prompt Construction: The app then guides the user in constructing the STAMP prompt for the chosen construct. The interface presents dedicated fields or templates for each component of the structured prompt: the construct definition, inclusion criteria, exclusion criteria, and exemplar examples. The user fills in each section (guidance and pre-filled examples are provided for reference). The app automatically formats these inputs into the structured prompt syntax (with the appropriate tags) required by the STAMP protocol. This guided prompt-builder ensures that even users unfamiliar with prompt engineering can create a robust, properly formatted prompt for their construct of interest.

3. Run STAMP Classification: Once the prompt is configured, the user initiates the classification run. For each piece of text data, the app sends the structured prompt along with the text instance to two different LLMs (by default, GPT-4.1 and a second top-performing model such as Gemini Flash 2 from Google DeepMind) in parallel. The models' classification outputs—including their step-by-step reasoning (if available)—are returned to the app. The platform automatically computes inter-model agreement statistics (Krippendorff's α and percent agreement) in real time as the data are being coded, giving the user immediate feedback on reliability.

4. Iterate Prompt if Needed: If the initial inter-LLM agreement falls below the desired threshold ($\alpha < .67$, indicating insufficient reliability), the app facilitates an iterative refinement process. It highlights the specific text instances where the two LLMs disagreed and uses a third LLM as an adjudicator to summarize common reasons for these disagreements (e.g., ambiguous wording, edge-case scenarios, misinterpreted criteria). Based on this feedback, the user can adjust the prompt by clarifying the construct definition, adding or refining inclusion/exclusion rules, or providing additional exemplars to address confusion. The user then re-runs the classification on the dataset with the revised prompt. This iteration loop can continue until agreement between the

models meets or exceeds the threshold (Krippendorff's $\alpha > .67$), indicating a reliably coded dataset.

5. Human Adjudication: Once inter-LLM reliability is achieved, any remaining individual instances of disagreement between the two LLMs are flagged for human review. For these cases, the app presents the text alongside the two models' outputs and rationales. A human coder (the user or a domain expert) can then examine the evidence and make the final determination for each disputed item. This ensures that the final labeled dataset resolves any subtle ambiguities that even the LLMs could not agree upon.

6. Results and Export: After classification (and human adjudication of any flagged cases), the app compiles the final set of labels for all texts. It also generates a summary report detailing the models' performance, including final agreement metrics and classification accuracy (if ground-truth labels or proxy measures like star ratings were available for comparison). The user can download the labeled dataset, the model-generated reasoning traces for each classification, and the summary report for documentation or further analysis.

By streamlining these steps, the web application lowers the barrier to implementing STAMP in both research and industry settings. Users can achieve reliable, valid construct classifications on their text data in a transparent and efficient manner, without requiring programming expertise. In essence, the tool operationalizes our framework and makes it accessible: marketing scholars can use it to analyze content with improved rigor and replicability, and practitioners (e.g., brand managers or analysts) can confidently deploy LLM-based content analysis knowing that the classifications are grounded in a validated protocol. This accessibility helps translate the methodological contributions of our work into real-world impact.

Online Appendix B: Limitations of Existing Text Classification Methods and STAMP's Advantages

STAMP introduces a paradigm shift in text-based construct classification by directly addressing key limitations of earlier approaches. Traditional methods—from dictionary-based content analysis to supervised machine learning to basic LLM prompting—each suffer gaps in reliability or validity. STAMP overcomes these gaps through an integrated, multi-agent architecture grounded in measurement theory. Below, we contrast STAMP with prior approaches:

Table B1. Comparing Extant Text Classification Methods (e.g., Dictionaries, Machine Learning) with STAMP

Dimension	Dictionary-Based Methods	Supervised ML (e.g., Supervised Vector Machine/Random Forrest)	STAMP Protocol (proposed)
Mechanism used	Static lexicons/keyword lists	Trained classifiers on labeled data	Multi-layer structured prompt, dual-LLM agreement + HITL adjudication
Data/label needs	None (but lexicon building is domain-specific)	High (large hand-labeled datasets needed for each construct/domain)	Low (zero-shot or few-shot; minimal exemplars needed) (Chae and Davidson 2025)
Context/polysemy handling	Weak (cannot disambiguate polysemous words or interpret context) (Humphreys and Wang 2018)	Dependent on the training domain and feature engineering	Strong (context-aware LLMs with structured prompts defining construct, criteria, and examples)
Transparency	Transparent (explicit word lists)	Opaque (black box with no direct explanation for predictions) (Villarroel Ordenes et al. 2025)	Transparent (chain-of-thought reasoning and structured outputs enable auditability)(Wei et al. 2022)
Validity	Face validity present; limited construct validity (Short et al. 2010)	Accuracy-focused; construct validity rarely evaluated (Humphreys and Wang 2018)	Built-in content, convergent, and discriminant validity checks

Dimension	Dictionary-Based Methods	Supervised ML (e.g., Supervised Vector Machine/Random Forrest)	STAMP Protocol (proposed)
			through multi-method comparisons (Cronbach and Meehl 1955)
Reliability	Inconsistent (brittle across domains; struggles with context) (Hartmann et al. 2023)	Cross-validation only; no inter-rater reliability analogy	High reliability via multiple independent LLM "raters" with agreement thresholds; disagreements resolved by human experts
Replicability/ standardization	Replicable within domain; brittle and domain-specific across contexts	Reproducible with code/data; not portable without retraining for new constructs	Highly portable and replicable standardized protocol across constructs and contexts
Scalability	High (once the lexicon is built)	Moderate (requires new labeled datasets for each application)	High (leverage pre-trained LLMs; no task-specific training required)
Key limitations	Cannot interpret context; misclassifies ambiguous cases	Requires extensive labeling; lacks transparency and validity checks; prone to hidden biases	Dependent on prompt quality, may struggle with entirely new theoretical constructs

Online Appendix C: Review of Prior Studies with Multiple Validation Methods

Below we present selected studies in marketing that use text analysis and report multiple validities to bolster confidence in their text-derived constructs. As Table C1 illustrates, there is heterogeneity in the reporting of the validity metrics. Specifically, no single study, to the best of our knowledge, reports all key forms of validity metrics (content, construct (convergent, discriminant), and predictive). This selective approach to validity reporting limits the conclusiveness and generalizability of reported methods. High cost and difficulty in collecting multiple new data sets or related measures needed to conduct comprehensive validity testing could be blamed. In contrast, STAMP's workflow makes it feasible to assess multiple facets of validity in one pipeline, as demonstrated in our studies.

Table C1. Selected Text Classification Studies Using Multiple Validity Checks

Research	Text Classification Method	Validity Type(s) Assessed
(Netzer et al. 2012)	Machine Learning- Unsupervised (topic modeling for Market Structure)	Convergent validity: ✓; Predictive validity: ✓
(Short et al. 2010)	Dictionary-based (custom word lists for Entrepreneurial Orientation)	Content validity: ✓; Discriminant validity: ✓; Predictive validity: ✓
(Ludwig et al. 2013)	Dictionary-based (LIWC for emotional tone + linguistic style matching algorithms)	Content validity: ✓; Predictive validity: ✓
(Timoshenko and Hauser 2019)	Machine Learning-Supervised (CNN classifier to filter & cluster online review sentences for customer needs)	Concurrent validity: ✓; Predictive validity: ✓
(Yeomans 2021)	Compare multiple methods (dictionary vs. domain-specific models for concreteness)	Construct validity: ✓; Predictive validity: ✓
(Martin and Kushwaha 2024)	Machine Learning (Top Management Team language (marketing vs. financial goals)	Construct validity: ✓; External validity: ✓
This Research	***STAMP Protocol***	***Content validity: ✓; Construct validity (convergent and discriminant): ✓; Predictive validity: ✓***

Online Appendix D: LLM Model Selection Choices and Cross-Model Generalizability

In developing STAMP, we carefully chose multiple LLMs and human-coding benchmarks to ensure rigorous evaluation. Two design considerations guided our LLM-LLM reliability setup: selecting heterogeneous model families and matching capability levels. We also standardized prompting across models and incorporated parallel human coding to benchmark reliability.

Diversity of Model Families

Using multiple models from the same LLM family can bias reliability results because models that share architecture, training data, or alignment procedures tend to correlate in their errors and agree more often than unrelated models. This can inflate apparent consensus (Liang et al. 2023)To mitigate this intra-family agreement and improve generalizability, our evaluation includes models from different providers. Specifically, we choose OpenAI (GPT-4.1 series), Google DeepMind (Gemini 2.x series), and Anthropic (Claude 3.5 & 4 series). This practice aligns with recommendations from holistic evaluation frameworks to assess AI systems across diverse model classes and prompting conditions(Liang et al. 2023, MMLU - Holistic Evaluation of Language Models (HELM)). Diversifying model families reduces the risk of an "echo-chamber" effect where models agree due to shared biases, yielding a more robust picture of inter-model consistency (Liang et al. 2023; Stanford CRFM n.d.).

D.1 Standardized Prompting Protocol for Model Benchmarking. All models are evaluated with a definition-only, zero-shot prompt (task definition and class labels, no exemplars) when establishing initial performance baselines. Zero-shot protocols are widely used in large-scale benchmarks to avoid heavy prompt engineering and to standardize comparisons across models (Hendrycks et al. 2021; Srivastava et al. 2022). While chain-of-thought prompting can boost scores, it introduces additional variability and reproducibility concerns (Wei et al. 2022). To mirror typical classification deployments and maintain reproducibility, we evaluate models in their standard inference modes without slow "thinking" variants or self-consistency voting (cf. OpenAI, 2025; Google, 2023; Wei et al., 2022). In practice, this means using straightforward prompts for each model uniformly and avoiding special tuning that could advantage one model over another.

D.2 Comparable Capability Levels. Models vary greatly in intelligence and cost. More expensive models may excel at tasks unrelated to text classification (e.g., multi-modal intelligence). To ensure a fair comparison, we select models at comparable levels of general intelligence using standardized benchmarks. We use Massive Multitask Language Understanding (MMLU) as an anchor for broad knowledge and reasoning (Hendrycks et al. 2021). MMLU assesses knowledge across 57 academic and professional subjects and is widely used for evaluating advanced LLMs. Because text classification depends heavily on language understanding, MMLU serves as a suitable benchmark for model selection. MMLU-Pro is a more challenging successor to MMLU, with more questions, ten answer options instead of four,

and stricter quality controls. Thus, we choose MMLU-Pro—a widely accepted language intelligence test for more advanced models such as Claude Sonnet 4.5.

Our model recommendation is based primarily on performance (similarity to human expert—crowd alpha, F1—crowd), followed by cost. We found that price does not predict performance on binary classification tasks. For example, low-cost models GPT-4.1-mini and Claude 3.5 Haiku cost much more than Google Gemini 2.X Flash but significantly underperform. Moreover, higher-intelligence and more expensive flagship models such as Claude 4.5 don't outperform GPT-4.1.

Table D1. Model Language Test and STAMP F1 Results

Model	MMLU-Pro	Avg F1 across three tactics	Precision and Recall Balance	Cost Tier	Recommendation
Claude 4.5 Sonnet	86%	0.860	Good	Expensive	Good alternative, but ceiling effect
GPT-4.1	81%	0.886	Excellent	Mid-range	Optimal flagship ✓
Gemini 2.5 Flash	81%	0.836	Good	Low-cost	Alternative to 2.0 Flash
Gemini 2.0 Flash	78%	0.863	Good	Low-cost	Optimal budget ✓
GPT-4.1-mini	78%	0.768	Poor (very low recall)	Low-cost	Avoid ✗
Claude 3.5 Haiku	63%	0.835	Poor (very low precision	Low-cost	Avoid ✗

Table D2. Model Cost Comparison

Model	Input Cost	Output Cost	Total (1M/1M)	Relative to GPT-4.1
Gemini 2.0 Flash	$0.10	$0.40	$0.50	Baseline (20× cheaper)
Gemini 2.5 Flash	$0.15	$0.60	$0.75	15× cheaper (but 50% more than 2.0)
GPT-4.1-mini	$0.40	$1.60	$2.00	5× cheaper (but poor performance)
Claude 3.5 Haiku	$0.80	$4.00	$4.80	2× cheaper (but poor performance)
GPT-4.1	$2.00	$8.00	$10.00	Reference
Claude 4.5 Sonnet	$3.00	$15.00	$18.00	1.8× more expensive

D.3 Generalizability and Robustness of Results Across Models. To enhance the robustness of our findings, we used bootstrap resampling (1,000 iterations) with temperature = 0 to produce deterministic output, rather than the default temperature of 1 used in ChatGPT for the main study. The results show that GPT-4.1 and Gemini 2.0 Flash perform indistinguishably from each other and outperform human experts in both reliability (alpha) and predictive validity (precision, recall, F1). We therefore focus on these two models in the main paper.

Table 3D. Inter-rater Reliability Between LLMs and Human Coders with Bootstrap Confidence Intervals

Construct & Prompt	Krippendorff's α (GPT-4.1 and Flash 2.0) [95% CI]	Disagreement Rate (GPT-4.1 and Flash 2.0)	Krippendorff's α (Human Expert / Crowd) [95% CI]	Disagreement Rate (Human Expert / Crowd)	Krippendorff's α (Trained RA / Crowd) [95% CI]	Disagreement Rate (Trained RA / Crowd)
Classic Luxury Style – Definition-Only	.42 [.30, .53]	6.41%	N/A	N/A	N/A	N/A
Classic Luxury Style – STAMP Prompt	.73 [.65, .80]	4.30%	.80 [.72, .87] ✓	2.80%	.69 [.60, .78]	4.00%
Personalized Service – Definition-Only	.67 [.60, .74]	7.88%	N/A	N/A	N/A	N/A
Personalized Service – STAMP Prompt	.86 [.81, .92] ✓	2.30%	.90 [.84, .94] ✓	1.50%	.79 [.71, .86] ✓	3.00%
Recommendation – Definition-Only	.46 [.38, .53]	23.87%	N/A	N/A	N/A	N/A
Recommendation – STAMP Prompt	.72 [.66, .78]	13.00%	.70 [.65, .76]	14.00%	.47 [.39, .54]	24.83%

Note: Bootstrap confidence intervals calculated using 1,000 replicates with percentile method (2.5th and 97.5th percentiles). ✓ indicates cells where the lower bound of the 95% CI meets or exceeds the 0.67 threshold, demonstrating statistical confidence in achieving acceptable reliability (Krippendorff, 2004). Human Expert / Crowd comparison is used as the primary benchmark for human-human performance. RA denotes Research Assistant. Flash 2.0 refers to Gemini 2.0 Flash.

Table 4D. STAMP Classification Performance Metrics with Bootstrap Confidence Intervals

Model / Rater	Metric	Classic Luxury Style	Personalized Service	Recommendation
GPT-4.1 (Flagship AI)	F1 / Precision / Recall	.83 [.77, .89] ✓ .82 [.73, .90] .84 [.77, .91]	.91 [.87, .96] ✓ .93 [.87, .98] .90 [.84, .96]	.91 [.89, .93] ✓ .89 [.86, .92] .94 [.91, .96]
Claude 4.5 Sonnet (Flagship AI)	F1 / Precision / Recall	.81 [.75, .87] ✓ .75 [.67, .84] .87 [.80, .94]	.90 [.85, .94] ✓ .83 [.76, .91] .98 [.94, 1.0]	.87 [.84, .90] ✓ .89 [.86, .92] .85 [.82, .89]
Gemini 2.0 Flash (Low-cost AI)	F1 / Precision / Recall	.82 [.76, .88] ✓ .82 [.73, .89] .83 [.75, .90]	.88 [.83, .93] ✓ .81 [.72, .88] .98 [.94, 1.0]	.88 [.86, .90] ✓ .83 [.79, .86] .94 [.92, .97]
Gemini 2.5 Flash (Low-cost AI)	F1 / Precision / Recall	.76 [.68, .83] .78 [.68, .87] .75 [.65, .83]	.88 [.82, .93] ✓ .80 [.72, .87] .98 [.94, 1.0]	.86 [.84, .89] ✓ .80 [.76, .84] .94 [.91, .96]
Claude Haiku 3.5 (Low-cost AI)	F1 / Precision / Recall	.81 [.73, .86] ✓ **.72 [.64, .80]** .91 [.84, .96]	.86 [.80, .90] ✓ **.79 [.70, .86]** .94 [.89, .99]	.84 [.82, .87] ✓ **.74 [.70, .79]** .98 [.96, .99]
GPT-4.1-mini (Low-cost AI)	F1 / Precision / Recall	.61 [.50, .71] .98 [.92, 1.0] **.45 [.34, .55]**	.84 [.76, .89] ✓ .95 [.90, 1.0] **.74 [.64, .83]**	.85 [.82, .88] ✓ .94 [.91, .96] **.78 [.74, .83]**
Crowd (Modal) (Human Benchmark)	F1 / Precision / Recall	.82 [.75, .88] ✓ .94 [.88, .99] .72 [.63, .82]	.91 [.85, .95] ✓ .94 [.87, .98] .88 [.80, .95]	.89 [.86, .91] ✓ .84 [.80, .87] .94 [.92, .97]
Trained RA (Human Benchmark)	F1 / Precision / Recall	.76 [.68, .83] .84 [.75, .92] .70 [.60, .79]	.86 [.79, .91] ✓ .88 [.81, .95] .83 [.75, .91]	.86 [.83, .89] ✓ .84 [.81, .88] .88 [.85, .91]

Note: Bootstrap confidence intervals calculated using 1,000 replicates with percentile method (2.5th and 97.5th percentiles). F1 ≥ 0.80 (good performance). ✓ = F1 ≥ 0.80. Bold indicates problematic precision or recall. The STAMP is revised one round by the researcher. The RA were trained for one to two rounds.

Online Appendix E: LLM-based Classification Methods

The emergence of LLMs has created opportunities for automated text classification in marketing research, yet early implementations have revealed critical methodological gaps that undermine the reliability and validity of the text constructs. Our analysis of current LLM applications identifies five fundamental limitations that necessitate the development of a standardized methodology.

Table E1. Comparison of LLM-based Text Classification Methods

Dimension	**Extant LLM-based Text Classification**	**STAMP Protocol (*advantages*)**
Prompting framework	Unstructured one-off prompts: results are prompt-sensitive, difficult to replicate across contexts (Liu et al. 2023, Schulhoff et al. 2025, Blanchard et al. 2025)	Multi-layer standardized prompt (definition, inclusion/exclusion exemplars) with template for input and output. (*Reduces brittleness, improves replicability/consistency across contexts*).
Measurement-theory integration	Typically, not integrated; lacks rigor of established frameworks(Cronbach and Meehl 1955, Churchill 1979); insufficient construct validation checks (MacKenzie et al. 2011)	Integrated in the prompt, checks for content, convergent, and discriminant validity. (*Allows assessment of construct validity*).
Single/Multiple LLM Use	Single LLM use, no way to assess the difficulty/uncertainty faced by the LLM during classification; simple treatment of multiple outputs without deeper analysis (Ziems et al. 2024)	Dual-LLM use with human adjudication for disagreement cases allows for prompt refinement based on disagreement patterns, analogous to inter-coder reliability (Artstein and Poesio 2008). (*Ambiguous cases are recognized, boost reliability, and focus human effort where it matters*).
Evaluation	Same model classifies as well as judges, creating AI-AI bias ; position and verbosity biases present ; no cross-model robustness checks, version/provider dependent (Zheng et al. 2023, Gao et al. 2025, Laurito et al. 2025)	Heterogeneous models are required to agree before classification; reliability thresholds ($\alpha \geq .67$) before acceptance. (*Mitigates model-specific idiosyncrasies and evaluation bias and enhances generalizability*).
Transparency and auditability	Limited reasoning for why decisions were made.	Structured rationales (captured for internal audit), explicit decision rules. (*Increases interpretability and post-hoc audit capacity*).

STAMP Design Elaboration

To address the above challenges, we introduce a structured prompting methodology that incorporates multiple layers of guidance to the LLM, advanced prompt engineering techniques, and measurement-theoretic principles. This design – referred to as the STAMP framework – is built on the following components and operationalized in the web-based application.

E.1 Multi-layer Prompt Architecture.

Construct Definition. The first layer of our prompt provides a clear and precise definition of the target construct, grounded in marketing theory. We explicitly state the conceptual boundaries of the construct, situating it in context so that the model has a solid foundation. For example, if the construct is "authenticity," the prompt begins with a formal definition of authenticity in consumer behavior, including its key characteristics. This ensures the LLM starts with a shared understanding close to that of a human expert. We also include domain contextualization in this layer, linking the construct to relevant domain knowledge or frameworks (e.g. noting if it relates to a known theory or established dimension in marketing literature). By disambiguating the construct from related ideas, the definition layer prevents the model from conflating it with superficially similar concepts. This practice aligns with best practice in measure development: just as researchers carefully define constructs before creating survey items(Churchill 1979), we give the LLM a proper theoretical definition up front. The result is improved face validity – the classification criteria look like they map to the construct of interest, because they stem from an expert-vetted definition (Cronbach and Meehl 1955). In short, Layer 1 acts as the "conceptual contract" for the model, anchoring all subsequent reasoning in a correct understanding of what is being measured.

Inclusion/Exclusion Criteria. The second layer enumerates specific criteria for identifying the construct in text, as well as clear exclusions. Here, we translate the abstract definition into operational guidelines that the LLM can apply. The prompt provides a bullet-point list or structured description of inclusion criteria – conditions or signals in the text that would indicate the presence of the construct. Likewise, it lists exclusion criteria, describing patterns that do not qualify or that belong to different constructs often mistaken for the target. For instance, for authenticity, inclusion criteria might include "the speaker recounts a personal experience truthfully, even if it's self-disparaging," whereas exclusion criteria might note "mere mentions of product quality or factual statements are not authenticity." These criteria are derived from literature and expert knowledge, effectively functioning as a codebook for the LLM. This layer injects measurement theory integration directly into the prompt: it mirrors how human coders are trained with codebooks and decision rules. By systematically specifying what counts vs. what does not, we ensure content validity – the prompt covers the full domain of the construct and sets boundaries, reducing ambiguity. We drew on established frameworks like Churchill's paradigm and Rossiter's C-OAR-SE procedure in formulating criteria (Churchill 1979, Rossiter 2002)For example, Rossiter (2002) emphasizes defining the object and attribute of a construct and enumerating key facets; our inclusion/exclusion lists serve that function for the LLM, breaking the construct into checkable components. This layer directly tackles the earlier lack of operational definitions: rather than leaving the LLM to infer its own criteria (which may be inconsistent), we program the criteria into the prompt. The benefit is two-fold: the LLM's

classifications become more consistent with theoretical intent, and we gain transparency into the basis for decisions (since we know what rules the model is supposed to follow).

Exemplar-based Guidance. The third layer of the structured prompt provides illustrative examples to solidify the model's understanding and guide its application of the criteria. We supply the LLM with a set of labeled examples – some texts that should be classified as the construct (positive examples) and some that should not (negative examples) – along with explanations for each. These exemplars serve as a form of few-shot training within the prompt (Brown et al. 2020). Crucially, the examples are chosen to cover a balanced representation of edge cases and typical cases. For instance, a positive example might be a tricky borderline case that just meets the inclusion criteria, with an explanation highlighting the feature that makes it qualify. A negative example might be a common pitfall (something that seems like the construct but fails the criteria) with an explanation clarifying why it is excluded. By seeing these examples, the LLM can better generalize the pattern of what is in or out of the construct. The inclusion of rationales ("explain why this text is or isn't an example of the construct") further enhances learning – it forces the model to connect the definition and criteria to real text instances in a step-by-step manner. This layer greatly improves the model's reasoning transparency: the model is effectively taught not just what the answer is for each example, but why that answer is correct, mirroring the logic an expert would use. Providing exemplars with explanations is known to boost model performance on classification tasks by giving concrete reference points (Brown et al. 2020, Liu et al. 2023). In our framework, it also addresses the earlier "black box" concern: since the model's decision process is shaped by explicit examples and reasoning, its eventual outputs can be more easily traced back to these known reference cases. Overall, Layer 3 functions as on-the-fly training data, embedding human-like judgment examples directly into the prompt. This drastically reduces ambiguity for the LLM and ensures that even borderline cases are handled in line with theoretical expectations.

Together, these three layers – Definition, Criteria, and Guided Examples – form a comprehensive, structured prompt that acts almost like a mini-expert system coded in natural language. Rather than asking the model a single blunt question (as early approaches did), we provide a fully specified framework for it to follow. This structured prompt is presented to the LLM in a consistent format for every classification task, thereby standardizing the methodology across constructs. By design, each layer addresses specific limitations identified earlier: Layer 1 counters vague construct understanding, Layer 2 prevents inconsistent or invalid classification by enforcing rules, and Layer 3 provides learning and transparency to overcome black-box behavior. The result is a prompt architecture that significantly improves reliability and validity in LLM-based text classification.

E.2 Engineered Structured Prompt Formatting. Implementing the multi-layer prompt architecture effectively requires strategic formatting. We incorporate several advanced prompt engineering techniques to ensure the LLM interprets and executes the structured prompt as intended.

XML-Structured Formatting. We format the prompt using a structured, markup-style syntax (inspired by XML) to delineate each section clearly. For example, the prompt might explicitly label sections with tags like <definition>...</definition>, <inclusion_criteria>...</inclusion_criteria>, <exclusion_criteria>...</exclusion_criteria>, and

<examples>...</examples>. This provides a semantic organization of the prompt content, making it absolutely clear to the model where one type of information ends and the next begins. Such structured formatting has practical benefits for both the model and any downstream processing. For the model, consistent tags reduce confusion – the LLM can learn that anything inside <inclusion_criteria> is a list of things that indicate the construct, for instance. This reduces the cognitive load on the model to parse our instructions. For downstream use, we often ask the LLM to produce its answer in a JSON or similarly structured format that mirrors the prompt structure. The parser-friendly output ensures that the LLM's response (which might include its classification decision and reasoning) can be easily extracted by software without errors. By minimizing free-form text in critical parts of the output and using fixed field labels, we achieve reduced parsing errors – the classification pipeline doesn't break due to unpredictable phrasing. Major AI providers have advocated for this kind of structured prompting as a best practice for reliability(Google Cloud 2025, Anthropic). In line with those guidelines, our use of an XML-like prompt template yields more consistent model behavior and facilitates automation. It essentially treats the prompt-response interaction as a programmatic interface, where the structured prompt is the "code" and the model's output can be read as structured data. This technique directly combats prompt brittleness: the model receives a standardized, clear instruction format each time, reducing variability due to prompt phrasing.

Chain-of-Thought Integration. To further enhance the model's reasoning process, we integrate chain-of-thought prompting (Wei et al. 2022)into our approach. This means the prompt explicitly instructs or allows the model to generate a step-by-step reasoning path before arriving at a final classification. In practice, after the structured prompt content, we might add: "Explain your reasoning step by step, then output the final classification." During inference, the model will first produce a reasoning trace (often in a designated section or tagged format), followed by its decision. Requiring the model to articulate a rationale serves several purposes. It forces the model to use the criteria and examples provided in a logical manner – rather than jumping to a conclusion, it must justify that conclusion, which tends to catch inconsistencies or misapplications of the rules. It also provides rationale storage: we retain these generated reasoning traces for each classification. These traces are invaluable for auditability and quality control. If the model makes an incorrect classification, we can inspect its own explanation to see where its logic went astray (e.g., did it neglect an exclusion criterion or misinterpret a keyword?). This helps in debugging the prompt or identifying areas where the model might need additional guidance. Moreover, by evaluating the reasoning traces, we can perform reasoning evaluation – checking if the model's thought process is sound and aligned with the construct's theory. If not, that signals a need for prompt adjustments. Prior work has shown that chain-of-thought prompting can significantly improve performance on complex reasoning tasks by allowing the model to break down problems (Wei et al. 2022)In our context, it improves classification accuracy and transparency simultaneously. Notably, Goli and Singh (2024) found that prompting GPT-4 to explain its decisions (a form of chain-of-thought prompting) mitigated some discrepancies between LLM outputs and human logic in a conjoint analysis task. We extend this idea by making it a standard part of our framework for every classification: every model decision comes with an explanatory justification. This approach transforms the LLM from a black box that mysteriously labels text into something more akin to a human coder that can "show its work" – a key for building trust in AI-driven measures.

Online Appendix F: STAMP Workflow (Prompts, Disagreements, Prompt Iterations) for Construct Classification

F.1 Study 1

Study 1 focused on employee-generated social media content in a high-context luxury apparel brand environment (Instagram posts by sales professionals). Two theoretical constructs were coded from these posts: a "brand–product" dimension capturing references to classic, timeless luxury style, and a "brand–service" dimension capturing personalized service or product customization. We provide below the full structured prompts used for each construct, examples of model outputs, and an analysis of inter-LLM disagreements (theme categorizations and representative cases). These details complement the main text's summary of reliability and validity results for Study 1.

F.1.1 "Classic Luxury Style" Construct

Full Structured Prompt (Study 1, Construct: Classic Luxury Style)

This prompt was designed as a codebook for identifying whether a social media post highlights the focal brand's classic, timeless, and refined style. The multi-layer prompt explicitly enumerates inclusion criteria (various forms of style emphasis) and exclusion criteria (cases focusing on unrelated aspects like material properties or casual style). An excerpt of the prompt is shown below (with formatting analogous to how it was input to the LLM):

You are a text classifier to identify the tactic "Classic Luxury Style": content about the classic, timeless, refined style of the focal luxury brand's clothing.

You will be provided with two pieces of information: <context> and <statement>. The "statement" contains a social media post about the focal luxury brand's clothing. Your task is to determine if the "statement" qualifies as the tactic based on the following criteria:

Inclusion Criteria (if ANY are true; it's the tactic):
1. Classic style emphasis – implicit or explicit language suggesting that the brand's clothing has a traditional, classic, professional, conservative, or similar style (e.g., "Classic never goes out of style", "Can't ever go wrong with a classic", "A double-breasted navy blazer is always a good choice.").
2. Timeless style emphasis – implicit or explicit language suggesting that the brand's clothing has a timeless, enduring, memorable, or similar style (e.g., "never go out of style", "tux that can be timeless").
3. Luxury style emphasis – implicit or explicit language suggesting that the brand's clothing has a luxurious, chic, hand-crafted, refined, elegant, sophisticated, fancy, stunning, gorgeous, or similar aesthetic (e.g., "Still refined, yet fresh", "all the luxury vibes", "Is there anything more elegant and sophisticated than a boucle tweed fabric?!?").

Exclusion Criteria (if ANY are true; it's NOT the tactic):
1. Material properties focus – **ONLY** discussion of material qualities such as fabric, color, or cut *without* a connection to classic, timeless, or luxurious style (e.g., "wool with cashmere

jacketing fabrics", "Pair this jacket with a navy flannel!").
2. Casual aesthetic appeal – **ONLY** emphasizes a casual, relaxed, or informal style without an explicit connection to classic, timeless, or luxurious aesthetics (e.g., "comfortable casual look," "laid-back style," "everyday casual appeal").
3. Non-text content only – statement that contains only punctuation or other non-textual characters.

Please analyze the given statement and provide your output in the following JSON format:

{
"reasoning": "inclusion_criteria [list the criteria number] and exclusion_criteria [criteria number]; explanation: [Your step-by-step reasoning, highlighting keywords. If you're uncertain about your decision, indicate 'uncertain'.]",
"tactic_prediction": [0 or 1]
}

<examples>

<example>

Context: "I am always seeking ways to set myself apart while remaining classic and conservative. Light gray is the answer to that sea of blue. Still refined, yet fresh."

Statement to detect: "I am always seeking ways to set myself apart while remaining classic and conservative."

{
"reasoning": "inclusion_criteria [1, 3] and exclusion_criteria []; explanation: The statement emphasizes 'classic and conservative' style.",
"tactic_prediction": 1
}
</example>

<example>

Context: "Consider going custom with a classic black suit or tux that can be timeless for all your formal events going forward! An absolute treat to wear and a treasure you'll keep for a lifetime."

Statement to detect: "Consider going custom with a classic black suit or tux that can be timeless for all your formal events going forward!"

{ "reasoning": "inclusion_criteria [1, 2] and exclusion_criteria []; explanation: The statement uses 'classic' and 'timeless' to describe the suit, emphasizing enduring style and a classic aesthetic appeal.",
"tactic_prediction": 1 }
</example>

<example>

Context: "This classic, lightweight linen blazer is a perfect staple for a comfortable, casual summer wardrobe."

Statement to detect: "This classic, lightweight linen blazer is a perfect staple for a comfortable, casual summer wardrobe."

{ "reasoning": "inclusion_criteria [1] and exclusion_criteria []; explanation: The statement directly mentions that the blazer is 'classic' even though it also mentions it as a staple piece in a casual wardrobe.",
  "tactic_prediction": 1 }
</example>

<example>

Context: "Check out this plum blazer on my client Tasha! She is a star in the board room!"

Statement to detect: "Check out this plum blazer on my client Tasha!"

{
  "reasoning": "inclusion_criteria [] and exclusion_criteria [1]; explanation: The statement only describes that the blazer is a plum color, a material property, without suggesting that it is classic, timeless, or luxurious.",
  "tactic_prediction": 0
}
</example>

<example>

Context: "All of our belts are made of the finest Italian leather and hand-crafted by highly skilled craftsmen. Call me today for a personal wardrobe consultation!"

Statement to detect: "All of our belts are made of the finest Italian leather and hand-crafted by highly skilled craftsmen."

{
  "reasoning": "inclusion_criteria [3] and exclusion_criteria []; explanation: The statement mentions that the belts are \"hand-crafted\" and made of the \"finest\" leather, which both indicate luxury quality.",
  "tactic_prediction": 1
}
</example>

<example>

Context: "Our new cashmere sweater is made of the softest material you will ever feel."

Statement to detect: "Our new cashmere sweater is made of the softest material you will ever feel."

{
  "reasoning": "inclusion_criteria [] and exclusion_criteria [1]; explanation: The statement only discusses material properties ('softest material') without any mention of classic, timeless, or luxurious style.",
  "tactic_prediction": 0
}
</example>

<example>

Context: "Made with the finest fabrics that are lightweight and breathable."

Statement to detect: "Made with the finest fabrics that are lightweight and breathable."

{
  "reasoning": "inclusion_criteria [] and exclusion_criteria [1]; explanation: The statement focuses exclusively on fabric qualities and physical properties ('lightweight and breathable') without referencing classic, timeless, or luxury style.",
  "tactic_prediction": 0
}
</example>

</examples>

Model Outputs and Agreement: Using the structured prompt, GPT-4.1 and Gemini Flash 2 coded 1,000 luxury post sentences in parallel. The final inter-model agreement for this construct was Krippendorff's $\alpha = .75$ (substantial agreement), with approximately 8% of items flagged for human adjudication (remaining disagreements). The disagreement themes between the two LLMs were analyzed to inform prompt refinement. Below, we summarize the main categories of disagreement for the Classic Luxury Style construct, along with illustrative examples of each.

Disagreement Themes (Structured Prompt, Classic Luxury Style):

Theme 1: Interpretation of Implicit vs. Explicit Style References (21 disagreements) – One model required explicit keywords ("classic," "timeless") while the other inferred classic style from context.

Sample cases: "Sunday best" – LLM2 missed that this idiom implies one's finest formal attire (classic style), which LLM1 correctly recognized. *"make the impression that you mean business"* – LLM1 understood this as implying a professional, traditional aesthetic (classic style) whereas LLM2 did not connect it to the classic style concept. *"boardroom"* – LLM1 picked up that this context implies formal, classic style expectations, but LLM2 failed to.

Theme 2: Understanding of Fashion/Style Terminology (8 disagreements) – One model showed more domain knowledge of fashion terms indicating classic style, where the other did not.
Sample cases: "peacoat" – LLM1 knew a peacoat is a classic garment and caught the classic style implication, while LLM2 did not. *"power suit"* – LLM1 understood this term connotes a timeless, professional style; LLM2 missed that connection. *"The power of a 3-piece suit!"* – LLM1 recognized cultural connotations of a three-piece suit (timeless, refined), whereas LLM2 did not.

Theme 3: Material Properties vs. Style Characteristics (4 disagreements) – One model sometimes mistook descriptions of materials or quality as indications of luxury style, which the other model correctly ruled out if style words were absent.
Sample cases: "beautiful luster" – LLM2 incorrectly took this as a luxury aesthetic signal, while LLM1 correctly noted it's a material property description. *"fancy yarn"* – LLM2 misinterpreted this technical term as indicating luxury, but LLM1 recognized it as a production term (not a style descriptor). *"hand-made"* – LLM2 assumed anything hand-made implies luxury; LLM1 required explicit luxury context to count it.

Theme 4: Context Misapplication (3 disagreements) – In a few cases, one model applied inclusion criteria out of context or produced irrelevant output, whereas the other model handled the context correctly.
Sample cases: In one instance, LLM1 encountered an API error and gave no result (flagged as disagreement). In another, LLM2 labeled a statement as luxurious (inclusion criterion [3]) without justification, whereas LLM1 correctly identified it as generic. *"exclusive... finest values"* – LLM2 misread "finest values" as luxury style, but LLM1 correctly interpreted it as referring to deals/pricing (not style).

Prompt Iteration: The above disagreement analysis led to minor prompt adjustments (e.g., clarifying that idioms implying formality count as classic style, and that "hand-made" alone doesn't guarantee luxury style without further context). After iteration, model agreement improved slightly (α up to .78) and fewer cases (~5%) required human adjudication.

(For thoroughness, a variant prompt was also tested that included an LLM-to-LLM reconciliation step: one model's output was given as additional context to the other to see if disagreements could be resolved automatically. While this "cross-talk" prompt slightly reduced trivial inconsistencies, it risked one model unduly influencing the other and was not used in the final protocol, in order to preserve independent judgments.)

F.1.2. Personalized Service Construct

Full Structured Prompt (Study 1, Construct: Personalized Service) – This prompt targets whether a post indicates that the brand customizes products or services to individual clients' needs or preferences. It delineates multiple inclusion rules (explicit mentions of custom offerings, one-to-one service experiences, emphasis on individual fit) and exclusion rules (generic marketing features, non-personalized invites, etc.). Key portions of the prompt are shown below:

You are a text classifier to identify the tactic "Personalized Service": content showing that a brand customizes products or services to an individual client's needs, desires, lifestyle, or preferences.

You will be provided with two pieces of information: <context> and <statement>. Your task is to determine if the "statement" qualifies as the tactic based on the following criteria:

Inclusion Criteria (if ANY are true; it's the tactic):
1. **Tailor-made offering** – mentions *custom, bespoke, made-to-measure, monograms* or other wording that indicates the product/service itself is created to a client's specifications.
2. **Explicitly personalized experience** – describes a one-to-one, private, or individualized shopping/delivery interaction (e.g., "private Zoom fitting," "one-on-one styling session," "house call tailoring," "personal wardrobe consultation").
3. **Emphasis on individual fit** – stresses that the offer adapts to the client's unique needs, desires, lifestyle, measurements, or preferences (e.g., "engineered for your lifestyle").

Exclusion Criteria (if ANY are true; it's not the tactic):
1. **Standard retail variety** – ONLY discussion of unique fabrics, wide product ranges, or other features that do **not** involve tailoring to the *individual*.
2. **Generic invite without personalization** – ONLY broad calls such as "DM me for a consultation" or "Let's hop on a call" that lack any wording implying privacy, individuality, or customization.
3. **Non-text content only** – statement that contains only punctuation or other non-textual characters.

Please analyze the given statement and provide your output in the following JSON format:

{{
"reasoning": "inclusion_criteria [list the criteria number] and exclusion_criteria [criteria number]; explanation: [Your step by step reasoning, highlighting keywords]",
"tactic_prediction": [0 or 1]
}}

<examples>

<example>

Context: "Don't rent a tuxedo when you could have one made for you! Wear a personalized custom formal blazer made strictly for you and blow the crowd away!"

Statement to detect: "Don't rent a tuxedo when you could have one made for you! Wear a personalized custom formal blazer made strictly for you and blow the crowd away!"

{{
"reasoning": "inclusion_criteria [1,3] and exclusion_criteria []; explanation: The statement emphasizes 'personalized custom' and 'made strictly for you', clearly indicating products tailored to individual needs.",
"tactic_prediction": 1
}}
</example>

<example>

Context: "Unlike box subscriptions & online styling services, I can make house calls & offer custom clothing right to your door."

Statement to detect: "Unlike box subscriptions & online styling services, I can make house calls & offer custom clothing right to your door."

{{
"reasoning": "inclusion_criteria [1,2] and exclusion_criteria []; explanation: The statement mentions 'house calls' (personalized service delivery) and 'custom clothing', indicating individualized service and products.",
"tactic_prediction": 1
}}
</example>

<example>

Context: "I save my clients time by coming to them in their offices or their homes. I assess what you do, who you see, where you travel. Based on that information I make suggestions on cloth & construction that will best suit your needs."

Statement to detect: "I save my clients time by coming to them in their offices or their homes... Based on that information I make suggestions on cloth & construction that will best suit your needs."

{{
"reasoning": "inclusion_criteria [2,3] and exclusion_criteria []; explanation: The statement describes personalized service delivery (house/office visits), individual assessment, and customization based on client's specific lifestyle and needs.",
"tactic_prediction": 1

}}
</example>

<example>

Context: "Personalized however you want! That's my favorite part about custom & who doesn't love clothing that fits?"

Statement to detect: "Personalized however you want! That's my favorite part about custom & who doesn't love clothing that fits?"

{{
  "reasoning": "inclusion_criteria [1,3] and exclusion_criteria []; explanation: The statement explicitly uses 'Personalized however you want' and mentions 'custom', directly indicating individualized products and services.",
  "tactic_prediction": 1
}}
</example>

<example>

Context: "Browse unique one-of-a-kind fabrics along with the last pieces of our best selling fabrics to complement your wardrobe."

Statement to detect: "Browse unique one-of-a-kind fabrics along with the last pieces of our best selling fabrics to complement your wardrobe."

{{
  "reasoning": "inclusion_criteria [] and exclusion_criteria [1]; explanation: The statement only discusses unique fabrics and product variety without mentioning any personalization, customization, or individual tailoring aspects.",
  "tactic_prediction": 0
}}
</example>

<example>

Context: "Give mom the gift of luxury & the custom experience this Mother's Day!"

Statement to detect: "Give mom the gift of luxury & the custom experience this Mother's Day!"

{{
  "reasoning": "inclusion_criteria [1,3] and exclusion_criteria []; explanation: The statement mentions 'custom experience', indicating personalized service tailored to individual preferences.",
  "tactic_prediction": 1

}}
</example>

</examples>

Model Outputs and Agreement: With this prompt, the same two LLMs (GPT-4.1 and Gemini Flash 2) coded the luxury post data for personalized-service content. Inter-model agreement for this construct reached Krippendorff's α = .82, reflecting high consistency (notably higher than the zero-shot definition-only baseline for this construct, which yielded α = .55 in initial trials). Only around 5% of instances required human adjudication after one prompt refinement iteration. The disagreements that did occur fell into clear thematic categories, which we detail below to illustrate the models' confusions:

Disagreement Themes (Structured Prompt, Personalized Service):

Theme 1: Recognition of Explicit Personalization Keywords (11 cases) – One model sometimes missed obvious personalization terms that the other caught.
Examples: LLM1 failed to recognize *"custom clothing company"* as implying personalization, whereas LLM2 did. In another case, LLM2 correctly flagged *"Custom"* in a phrase ("Custom Comfort Casual line") while LLM1 overlooked it. Such misses were rare but pointed to differences in keyword sensitivity.

Theme 2: Implicit vs. Explicit Personalization (8 cases) – One model required direct mentions of personalization, while the other inferred it from context.
Examples: "creating his look" – LLM1 saw this as indicating a personalized service (styling a specific individual), but LLM2 argued it could be generic. *"fit wasn't right for him off the rack"* – LLM2 recognized this implies a need for personalization, while LLM1 focused only on the fit issue without drawing the personalization inference. *"put your own twist on it"* – LLM1 interpreted this as an invitation to customize, whereas LLM2 wanted a more explicit term.

Theme 3: General Product Features vs. Individual Customization (3 cases) – Confusion over whether describing product suitability for certain customers counts as personalization.
Examples: "great for clients that run warm" – LLM1 argued this shows adaptation to a client type (personalization), but LLM2 correctly noted it's just product positioning (a feature for a segment, not individual tailoring). *"better fit"* – LLM2 took this as personalization, but it was in context a general quality claim. *"lifestyle" references* – LLM1 sometimes claimed a generic lifestyle mention was adaptation, whereas LLM2 treated it as describing a target demographic.

After prompt clarification (e.g., emphasizing that broad targeting of segments isn't individual personalization), these discrepancies were resolved.

Summary of Structured-Prompt Performance: Overall, the structured STAMP prompts for Study 1 construct yielded high inter-LLM reliability (α =.75–.82) and low disagreement rates, matching the reliability of human coders. The careful definitions and examples helped the

models agree on nuanced content (e.g., what counts as “timeless style” or “personalized service”). In contrast, a definition-only prompt baseline (providing only a brief construct definition and the task) resulted in significantly lower agreement: for the personalized-service construct, the models’ $\alpha = .40–.55$ with frequent divergent interpretations. We analyzed those baseline disagreements as well, to highlight what the structured prompt added:

Disagreement Themes (Definition-Only Baseline, Personalized Service):

Using only a basic instruction (e.g., *“Decide if the statement shows the brand customizing its products or services to the customer’s needs”*), the models often disagreed due to lack of guidance:

Theme 1: Generic vs. Personalized Language (28 cases) - One model marked posts as personalized based on generic positive language, while the other required explicit customization cues. E.g., *“If you’re like me...”* – LLM2 mistakenly took a conversational opener as personalization, whereas LLM1 correctly did not. Both models lacked a clear rule here without the structured prompt.

Theme 2: Handling Empty/Missing Content (15 cases) - Without specified criteria, one model might try to categorize even irrelevant input. E.g., an input that was just "!" – LLM1 correctly refused to classify (no content), but LLM2 *“hallucinated”* finding words like "customizing" and erroneously made a judgment. The structured prompt explicitly prevented this by defining non-text exclusion.

Theme 3: Product Features vs. Customization (12 cases) - The definition-only prompt left ambiguity on whether product-feature mentions imply personalization. LLM2 often assumed *any* product feature (like “well-fitted suit”) signaled personal fit; LLM1 generally did not. Without explicit criteria, they diverged on these borderline cases.

Theme 4: Consultation Offers vs. Actual Personalization (8 cases) - Phrases like “Contact us for a consultation” caused confusion. One model (LLM2) tended to count any consultation offer as personalization, whereas the other (LLM1) was unsure. The structured prompt’s exclusion criterion clarified that generic invitations alone do not count.

Theme 5: Customer Choice vs. Brand Customization (6 cases) - The models disagreed if a post said a client *chose* something vs. the brand *custom-made* something. LLM2 sometimes conflated customer choice (e.g., “client picked out a style”) with personalization; LLM1 did not. Clear guidance was needed to distinguish these (which the structured prompt provided by focusing on brand actions).

Theme 6: Incomplete Analysis or Template Responses (4 cases) - The definition-only setting led to occasional failures like one model giving a templated or no answer (e.g., returning an error code or a placeholder). The structured approach with a fixed JSON format mitigated such cases by enforcing a response structure and rationale.

These issues illustrate how the multi-part STAMP prompt (construct definitions + criteria + examples) drove substantial improvements in reliability and interpretative alignment between models. Under the full STAMP prompt, many of the above ambiguities were resolved via explicit instructions—resulting in the high agreement reported for Study 1 in the main text.

F.1.3 Robustness to Model Choice and Prompting

We assess whether the personalization-tactic classifier is robust to (i) model choice within the same family, (ii) model choice across major families, and (iii) the newest model generation. We also test whether the STAMP prompting protocol improves agreement relative to a definition-only prompt.

Models and families. We cover the three major model families—OpenAI, Google Gemini, and Anthropic Claude—with Grok excluded as still emerging. Within OpenAI, we compare GPT-4.1 and GPT-4o; across families, we compare Claude Sonnet 4 and Gemini 2.5 Flash; and we benchmark OpenAI GPT-5 (Medium) against human researcher labels. All analyses focus on the personalization tactic. We found that:

- Within-family alignment is "mono-cultural." In other words, GPT-4.1 and GPT-4o yield near-identical classifications ($\alpha = .85$).

- Structured prompting drives cross-family generalization. Claude Sonnet 4 and Gemini's newer model (2.5 Flash) agreement is excellent with the STAMP prompt (F1 = 94%, $\alpha = .93$) but weak with definition-only (F1 = 67%, $\alpha = .62$).

- Newest model (GPT5) performed similarly to prior best text model GPT4.1 vs. humans. GPT-5 slightly outperforms GPT-4.1 against researcher labels.

- Methodological implication. Standardized full prompting is essential for high cross-family reliability; model provider matters less once prompting is standardized.

Table F1. Cross-model Agreement on the Personalization Tactic

Comparison	Prompt	F1	Disagreement	Krippendorff's α
GPT-4.1 vs GPT-4o	STAMP	.86	2.40%	.85
GPT-4.1 vs GPT-4o	Definition	.84	2.30%	.83
Claude Sonnet 4 vs Gemini 2.5 Flash	STAMP	.93	1.21%	.92
Claude Sonnet 4 vs Gemini 2.5 Flash	Definition	.67	9.49%	.62
Researcher vs GPT-5 (Medium)	STAMP	.92	1.21%	.91
Researcher vs GPT-4.1	STAMP	.92	1.30%	.91

F.2. Study 2

Study 2 examined a low-context setting: consumer product reviews (from an e-commerce platform). The focal construct was the presence of a “Recommendation” tactic in a review, meaning the reviewer either endorses or criticizes the product or makes comparisons to other products. We detail the prompt used and model performance for this construct.

F.2.1 “Recommendation” Construct

Full Structured Prompt (Study 2, Construct: Recommendation) – The prompt instructs the model to decide if a review statement contains a positive or negative endorsement, or a comparative evaluation relative to other products. Key components include criteria for Endorsements and Comparisons, with an exclusion for non-content (punctuation-only). An excerpt follows:

You are a text classifier to identify the tactic "Recommendation": content implicitly or explicitly showing that a review statement contains a positive or negative endorsement or comparisons with other products.

You will be provided with two pieces of information: <context> and <statement>. Your task is to determine if the "statement" qualifies as a recommendation based on the following criteria:

Inclusion Criteria (if ANY are true; it's the tactic):
1. **Endorsement** – mentions positive or negative evaluations (implicit or explicit) that show the reviewer's satisfaction or dissatisfaction.
2. **Comparisons** – describes comparisons with other products (e.g., "compared to other products, this is best...", "this was bigger than brand B...") that position the product relative to alternatives.

Exclusion Criteria (if ANY are true; it's NOT the tactic):
1. **punctuation only** – statement that contains only punctuation.

Please analyze the given statement and provide your output in the following JSON format:

{{
"reasoning": "inclusion_criteria [list the criteria number] and exclusion_criteria [criteria number]; explanation: [Your step by step reasoning, highlighting keywords]",
"tactic_prediction": [0 or 1]
}}

<examples>

<example>

Context: "Product review for a kitchen appliance"

Statement to detect: "Very satisfied with this purchase!"

{{
  "reasoning": "inclusion_criteria [1] and exclusion_criteria []; explanation: The statement clearly displays a positive endorsement with 'Very satisfied', showing the reviewer's own satisfaction.",
  "tactic_prediction": 1
}}
</example>

<example>

Context: "Product review for household item"

Statement to detect: "Great capacity and easy to clean"

{{
  "reasoning": "inclusion_criteria [2] and exclusion_criteria []; explanation: The statement talks about positive features and implicitly compares it against similar products by highlighting superior qualities (great capacity, easy to clean).",
  "tactic_prediction": 1
}}
</example>

<example>

Context: "Product review for home decor"

Statement to detect: "You will love it"

{{
  "reasoning": "inclusion_criteria [1] and exclusion_criteria []; explanation: The statement clearly displays a positive endorsement with 'You will love it', showing the reviewer is making a recommendation to the reader.",
  "tactic_prediction": 1
}}
</example>

</examples>

Model Outputs and Agreement: With this prompt, GPT-4.1 and Gemini Flash 2 coded 1,000 review snippets for the presence of recommendations/comparisons. Inter-model agreement was high (Krippendorff's $\alpha = .87$), and only ~3% of items required human reconciliation in the end. The structured prompt effectively cued the models to pick up on subtle positive/negative sentiment cues and comparative language. Below, we summarize the disagreement themes observed (with the structured prompt) and then contrast with a baseline prompt.

Disagreement Themes (Structured Prompt, Recommendation):

Theme 1: Recognition of Evaluative Language (47 cases) – In a few instances, one model overlooked subtle evaluative words that the other caught.
Samples: "fits perfectly" – LLM1 initially missed *"perfectly"* as a positive qualifier. *"biggest complaint"* – LLM1 missed the clear negative sentiment, whereas LLM2 caught it. *"Great price!"* – LLM1 failed to mark *"Great"* as a positive evaluation. These were mostly cases of LLM1 being slightly less sensitive to single-word exclamations; after noticing this, the prompt was adjusted or the model outputs were reviewed to correct such misses.

Theme 2: Implicit vs. Explicit Endorsement (31 cases) – One model required a direct recommendation phrasing, while the other inferred satisfaction from context.
Samples: "Don't have that problem anymore!" – LLM1 missed that this implies a positive outcome (implicitly endorsing the product for solving a problem), which LLM2 recognized. *"keeps your ride nice and clean"* – LLM1 read this as factual description, LLM2 saw it as an endorsement of a benefit. *"made it work for me"* – LLM1 missed that despite modifications, the user implies a positive resolution; LLM2 interpreted it as a qualified endorsement.

Theme 3: Comparative Language Interpretation (14 cases) – One model sometimes failed to catch implied comparisons or relative statements.
Samples: "larger than I would prefer" – LLM1 missed the implicit comparison to an ideal size, flagged by LLM2 as a (negative) comparison. *"most common size"* – LLM1 saw a mere description; LLM2 inferred an implicit comparison to other sizes (positioning this size as standard). *"anything larger would be excessive"* – LLM1 missed the comparative judgment implied; LLM2 caught it.

Theme 4: Factual Description vs. Endorsement (13 cases) – The models occasionally disagreed on whether a neutral statement carried evaluative meaning.
Samples: "easy to empty and keep clean" – LLM1 treated this as a factual description; LLM2 interpreted it as highlighting a benefit (positive endorsement). *"solid materials" and "definitely handle spills"* – LLM1 saw objective facts; LLM2 heard confidence/assurance (positive tone). *"environmentally friendly"* – LLM1 took it as a fact about the product; LLM2 saw it as a promotional positive attribute. (These fine differences underscore the importance of context; the structured prompt's examples helped align interpretations.)

Theme 5: Incomplete/Ambiguous Statements (5 cases) – For fragmentary or unclear snippets, one model would guess sentiment while the other abstained or flagged uncertainty.
Samples: "It is leak" – LLM1 recognized this as an incomplete fragment (probably "It is leaking" intended) and refrained from strong classification, whereas LLM2 assumed a negative meaning. *"Have no idea how leak"* – LLM1 noted ambiguity; LLM2 inferred a product defect

(negative). *"Not sure I would re"* – LLM1 saw a truncation and hesitated; LLM2 assumed it was leading to "not recommend" and took it as negative. In general, the structured prompt did not explicitly instruct how to handle incomplete sentences beyond the punctuation-only rule, so we relied on manual review for truly unclear cases.

Definition-Only Baseline vs. Structured Prompt: In a zero-shot baseline (asking the models simply "Does this statement contain a recommendation?"), the agreement was much lower (α =.60), and systematic discrepancies emerged:

Themes of Disagreement (Baseline): Without the detailed criteria, one model (often GPT-4.1) leaned cautiously, classifying only obvious cases of "I recommend" or "I love this" as positive endorsements, whereas the other model (Gemini Flash 2) tended to over-predict the presence of recommendations by counting any positive adjective as an endorsement. For example, "fits perfectly" or "works as advertised" were often labeled as recommendations by one model but not the other when no strict guidelines were given. The lack of an explicit *comparison* criterion also led to many missed or inconsistent detections of comparative language in the baseline. By contrast, in the structured STAMP prompt, both models were explicitly cued to look for comparisons (e.g., "bigger than X"), and we provided an example, which greatly increased consistency on those cases.

The structured prompt forced both models to output a step-by-step reasoning in JSON. This revealed when a model was guessing or misinterpreting. In baseline prompting, we had less visibility into *why* a model gave an answer, making it harder to reconcile disagreements or improve the prompt. With STAMP's chain-of-thought outputs, we observed, for instance, that one model might say *"no recommendation because this is just a feature description"* versus the other saying *"contains an implicit endorsement because feature is described positively."* This transparency enabled targeted prompt refinements.

In summary, Study 2's construct (recommendations) benefited substantially from STAMP's structured approach. The dual-LLM agreement reached human-parity levels (α in the high .80s, matching crowd vs. expert agreement for this task) and the models' combined outputs, after adjudication, achieved classification accuracy slightly exceeding that of individual human coders (as reported in the main text). The details provided above underscore how the structured prompt and dual-model setup resolved many ambiguities that a naive prompting strategy would struggle with.

Online Appendix G: Reliability Benchmarking

G.1 Inter-Rater Reliability (LLM-LLM and Human–Human). We assess inter-rater reliability by comparing agreement between LLM predictions and human judgments, using two human rating paths for rigor:

1. **Expert evaluator:** One PhD-level domain expert independently rates all items.
2. **Crowd consensus:** 4–11 trained independent crowd coders label each item; we aggregate their labels via majority vote or a comparable consensus rule.

This dual approach provides both a high-knowledge "gold" perspective and a realistic multi-rater baseline (cf. standard content-analysis practice). We compute agreement statistics such as Krippendorff's α for (a) expert vs. crowd consensus, (b) expert vs. each model, and (c) crowd consensus vs. each model. Krippendorff's α accounts for chance agreement and is widely recommended for content analysis and labeling studies (Gwet 2014, Krippendorff 2018).By applying the same reliability analyses to human–human pairs and LLM-LLM pairs, we can judge whether model–model disagreements are on the order of normal expert variability or if they diverge substantially beyond human norms.

G.2. Reliability Measures

We report Krippendorff's α as our primary reliability measure. Below we detail the calculations for Krippendorff's α.

Krippendorff's α is a flexible measure of inter-rater agreement and generalizes to any number of raters, missing data, and multiple levels of measurement (nominal, ordinal, interval, ratio). For nominal data:

$$\alpha = 1 - D_o / D_e$$

where:

D_o = observed disagreement, calculated as the sum of pairwise disagreements across all items

D_e = disagreement expected by chance, based on the marginal distribution of categories across all raters and items

More precisely, for nominal categories, D_o is the proportion of paired comparisons (within items) that fall into different categories, and D_e is the same quantity computed under the assumption that category assignments are independent of items.

Prior research has suggested the following cut-offs for Krippendorff's α. Specifically,

$\alpha \geq .80$: Acceptable for drawing definitive conclusions

$.67 \leq \alpha < .80$: Acceptable for tentative conclusions or exploratory research

$\alpha < 0.67$: Unacceptable, suggesting substantial disagreement

References

Anon MMLU - Holistic Evaluation of Language Models (HELM). Retrieved (October 13, 2025), https://crfm.stanford.edu/helm/mmlu/latest/.

Anthropic Use XML tags to structure your prompts. *Anthropic*. Retrieved (March 13, 2025), https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/use-xml-tags.

Artstein R, Poesio M (2008) Inter-Coder Agreement for Computational Linguistics. *Comput. Linguist.* 34(4):555–596.

Blanchard SJ, Duani N, Garvey AM, Netzer O, Oh TT (2025) New Tools, New Rules: A Practical Guide to Effective and Responsible Generative AI Use for Surveys and Experiments in Research. *J. Mark.* 89(6):119–139.

Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, et al. (2020) Language Models are Few-Shot Learners. *Adv. Neural Inf. Process. Syst.* 33:1877–1901.

Chae Y, Davidson T (2025) Large Language Models for Text Classification: From Zero-Shot Learning to Instruction-Tuning. *Sociol. Methods Res.*:00491241251325243.

Churchill GA (1979) A Paradigm for Developing Better Measures of Marketing Constructs. *J. Mark. Res.* 16(1):64.

Cronbach LJ, Meehl PE (1955) Construct validity in psychological tests. *Psychol. Bull.* 52(4):281.

Gao Y, Lee D, Burtch G, Fazelpour S (2025) Take Caution in Using LLMs as Human Surrogates: Scylla Ex Machina. (January 23) http://arxiv.org/abs/2410.19599.

Goli A, Singh A (2024) Frontiers: Can Large Language Models Capture Human Preferences? *Mark. Sci.* 43(4):709–722.

Google Cloud (2025) Structure prompts | Generative AI | Google Cloud. *Google Cloud*. Retrieved (September 24, 2025), https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/structure-prompts.

Gwet KL (2014) *Handbook of inter-rater reliability: the definitive guide to measuring the extent of agreement among raters* Fourth edition. (Advances Analytics, LLC, Gaithersburg, Md).

Hartmann J, Heitmann M, Siebert C, Schamp C (2023) More than a Feeling: Accuracy and Application of Sentiment Analysis. *Int. J. Res. Mark.* 40(1):75–87.

Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, Steinhardt J (2021) Measuring Massive Multitask Language Understanding. (January 12) http://arxiv.org/abs/2009.03300.

Humphreys A, Wang RJH (2018) Automated Text Analysis for Consumer Research Fischer E, Price L, eds. *J. Consum. Res.* 44(6):1274–1306.

Krippendorff K (2018) *Content analysis: An introduction to its methodology* (Sage publications).

Laurito W, Davis B, Grietzer P, Gavenčiak T, Böhm A, Kulveit J (2025) AI–AI bias: Large language models favor communications generated by large language models. *Proc. Natl. Acad. Sci.* 122(31):e2415697122.

Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Yian, et al. (2023) Holistic Evaluation of Language Models. (October 1) http://arxiv.org/abs/2211.09110.

Liu P, Yuan W, Fu J, Jiang Z, Hayashi H, Neubig G (2023) Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Comput. Surv.* 55(9):1–35.

Ludwig S, Ruyter KD, Friedman M, Brüggen EC, Wetzels M, Pfann G (2013) More Than Words : The Influence of Affective Content and Linguistic Style Matches in Online Reviews on Conversion Rates. *J. Mark.* 77(January):87–103.

MacKenzie, Podsakoff, Podsakoff (2011) Construct Measurement and Validation Procedures in MIS and Behavioral Research: Integrating New and Existing Techniques. *MIS Q.* 35(2):293.

Martin A, Kushwaha T (2024) EXPRESS: Can Words Speak Louder than Actions? Using Top Management Teams' Language to Predict Myopic Marketing Spending. *J. Mark.*:00222429241244804.

Netzer O, Feldman R, Goldenberg J, Fresko M (2012) Mine Your Own Business: Market-Structure Surveillance Through Text Mining. *Mark. Sci.* 31(3):521–543.

Rossiter JR (2002) The C-OAR-SE procedure for scale development in marketing. *Int. J. Res. Mark.* 19(4):305–335.

Schulhoff Sander, Ilie M, Balepur N, Kahadze K, Liu A, Si C, Li Y, et al. (2025) The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. (February 26) http://arxiv.org/abs/2406.06608.

Short JC, Broberg JC, Cogliser CC, Brigham KH (2010) Construct Validation Using Computer-Aided Text Analysis (CATA): An Illustration Using Entrepreneurial Orientation. *Organ. Res. Methods* 13(2):320–347.

Timoshenko A, Hauser JR (2019) Identifying customer needs from user-generated content. *Mark. Sci.* 38(1):1–20.

Villarroel Ordenes F, Packard G, Hartmann J, Proserpio D (2025) Using Traditional Text Analysis and Large Language Models in Service Failure and Recovery. *J. Serv. Res.* 28(3):382–388.

Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi EH, Le QV, Zhou D (2022) Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Adv. Neural Inf. Process. Syst.* 35:24824–24837.

Yeomans M (2021) A concrete example of construct construction in natural language. *Organ. Behav. Hum. Decis. Process.* 162:81–94.

Zheng L, Chiang WL, Sheng Y, Zhuang S, Wu Z, Zhuang Y, Lin Z, et al. (2023) Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. (December 24) http://arxiv.org/abs/2306.05685.

Ziems C, Held W, Shaikh O, Chen J, Zhang Z, Yang D (2024) Can Large Language Models Transform Computational Social Science? *Comput. Linguist.* 50(1):237–291.